

Portfolio Document

Prepared by Viktor Ljungberg, September 2026, with AI assistance from Claude (Anthropic).

Technical portfolio project for a 2026-2028 career transition into AI implementation, digitalisation and IT project coordination.

What this document is.

This is item 4 of the six criteria docs/PHASE7_NEXT.md sets for calling this project "portfolio-ready": the fourteen sections the career plan specifies, covering the engine end to end. Item 5, the separate AI-attribution accounting of what was designed, decided, tested and ruled on by Viktor versus what Claude produced, is assembled separately and is not part of this document. This document does not itself declare the release gate open, the remaining findings closed, or the project portfolio-ready -- those are Viktor's own calls, left open in the project's own record as of this writing.

Contents

1. Objective
2. Problem Definition
3. Architecture
4. Data Sources
5. Modules
6. Decision Logic
7. Testing Methodology
8. Debugging Process
9. Major Problems
10. Solutions
11. Results
12. Limitations
13. Lessons Learned
14. Future Development

1. Objective

Phase-7 is a structural quant engine: an analytical and decision-support tool that reads public market data for a given crypto pair and timeframe and produces a directional read (long, short, or no-trade), an entry price, three targets, a stop, and a risk verdict. It holds no exchange credentials, places no orders, and has no capacity to execute a trade or move money -- by design, not as a limitation to be lifted later. Its job is to inform a trading decision a person makes, not to make one itself.

The engine's governing document, the Phase-7 Engineering Constitution (ratified 26 August 2026), states this as Tier 0, above every other rule: *"The engine must never be optimized to appear intelligent. It must be optimized to produce reliable, testable, interpretable information and decisions under real-world conditions."* Every design choice in this project is measured against that line, including ones proposed by Claude and later reversed.

This is also the technical portfolio project for a 2026-2028 career move into AI implementation, digitalisation and IT project coordination -- not a step toward becoming a full-time programmer or trading professional. What it has to demonstrate is independent technical work: architecture, testing, debugging, iterative development, and structured decision-making under real constraints. It does not have to make money, and a validation result that comes back negative is a result, not a failure, provided the evidence for it is shown.

2. Problem Definition

Reading market structure by eye is inconsistent: the same chart looks different to the same person on different days, and nothing forces a trader to apply the same criteria twice. The problem this project addresses is building a system that applies one fixed, inspectable set of rules to every run -- the same structural logic, the same risk checks, the same refusal to invent a number it cannot support -- regardless of who is looking at the output or how the market has been behaving lately.

This is the third build of this engine. An earlier build was broken by backtesting work carried out before the project had version control, known-good checkpoints, or a fixed evaluation dataset to check against -- a change could not be verified safe, and a working system was lost as a result. That failure defines this build's engineering discipline as much as any trading logic does: roughly forty test runs and validations went into reaching the current version, and backtesting -- while still a planned, real priority -- is deliberately sequenced last, only after a proper safety net exists, so that a failed experiment stays a recoverable setback instead of a fourth rebuild.

The Constitution frames the stakes precisely: a confident-looking number computed from broken input, or a system that quietly becomes more certain than its evidence supports, is worse than no output at all, because it does not announce itself as wrong. The engineering problem, in other words, is not "can this system predict markets" -- it explicitly does not have to -- but "can this system be trusted to say only what it actually knows, and say so the same way every time."

3. Architecture

The engine runs as a single linear pipeline, entered through **main.py**, which loads configuration, then hands control to **SignalRouter.route()** (models/signal_router.py). Everything downstream of that call happens in one pass, for one symbol and timeframe, with no step allowed to reach back and change a decision an earlier step already made -- a run either produces a decision object or fails visibly, never a partial one presented as complete.

The pipeline itself lives mostly in core/engine_core.py's **Phase7Engine.run()**, which calls out to the other directories in a fixed order: data/data_fetcher.py fetches candles and validates them in the same call --

data/validation.py's **validate_ohlcv()** rejects missing candles, duplicates, impossible prices, bad timestamp ordering, and stale data before anything downstream ever sees the frame. indicators/indicators.py computes the technical indicators, structure/structure.py finds swing structure and volume nodes, indicators/trend_health.py scores trend health, and models/bias_engine.py turns all of that into the one directional read (**raw_bias**) that the rest of the run treats as authoritative -- a same-shaped computation runs a second time for BTC, so models/btc_context.py can score how correlated and stressed the wider market is without that correlation ever being allowed to cast a second, overriding vote on direction. models/entry_model.py scores entry quality, models/risk_model.py computes the stop and three targets, and models/decision_model.py assembles the final action, reading bias_engine's output as the sole source of direction rather than re-deriving it. core/panel_render.py renders the result for a human to read, and core/decision_log.py writes it to a permanent record.

Three modules exist specifically to make a run checkable after the fact rather than only readable at the time: core/lineage.py hashes every input frame and the run itself, archiving the raw candles so an old decision stays verifiable even once the exact market data behind it can no longer be re-fetched; core/code_fingerprint.py hashes the code that produced the decision, so a decision can be tied to the exact version of the engine that made it; and core/decision_contract.py defines the shape a decision object is required to have, and is checked against every run's output before that output is allowed to reach the panel or the log. live_trading.py is kept structurally separate from everything above -- it holds the engine's only market-facing code, and it is read-only, enforced by five separate guards rather than one, so that Tier 1 Item 18 (no trade-execution credentials) does not depend on nobody making a mistake in one place.

The project has 213 commits and a 51-file test suite, organized the same way the engine itself is: one test module per production concern, plus a set of pinned fixtures (tests/fixtures/) that let the whole pipeline be re-run against a known input and checked against a known-good output -- the golden path referenced throughout the engineering log.

4. Data Sources

The engine has exactly one external data source: MEXC's public REST API, called only through its unauthenticated /api/v3/klines endpoint. No API key or secret is ever declared or read anywhere in the codebase -- an earlier version of the configuration file had two empty credential slots for them, and those were removed rather than left unused, on the reasoning that an empty credential field sitting in the config of an engine that must never trade is an invitation, not a setting. This is the same invariant (Tier 1, Item 18 -- read-only market access) enforced twice: once architecturally, by keeping live_trading.py as the engine's only market-facing code, and once at the data layer, by the fact that nothing in the fetch path is even capable of authenticating.

A single run pulls three OHLCV series from that one source: the base symbol on its execution timeframe (AEROUUSD on 4h by default -- both are configuration, not hardcoded, so the engine runs unchanged against any pair MEXC lists), the same symbol on a macro higher timeframe (1d) for multi-timeframe confluence, and BTCUSD on the base timeframe, used only to score how correlated and stressed the wider market is, never to vote on direction itself. Every one of the three is checked by the same validator (data/validation.py) the moment it is fetched -- missing candles, duplicates, impossible prices, out-of-order timestamps, and stale data are all rejected before anything downstream can treat them as real, and each fetch returns its own independent copy of the frame rather than one shared object, closing a class of bug the remediation found where modules were rewriting a caller's columns in place.

Live data is, by nature, impossible to fetch twice identically, which made it impossible to satisfy the Constitution's reproducibility requirement -- changing one setting, rerunning, and comparing outputs cannot mean anything if the market also moved between the two runs. The fetcher's answer is a pinned source: pointed at a directory of CSVs

{{SYMBOL}}_{TIMEFRAME}.csv, one per series), it serves every fetch from disk instead of the network. tests/ uses this for a fixed synthetic pair, TESTUSDT, so the suite runs against the same candles every time regardless of what the real market is doing. A missing file under an active pinned source is treated as an error rather than a silent fall-through to the live API, deliberately -- falling back would quietly reintroduce the exact nondeterminism pinning exists to remove. Network calls that do reach MEXC carry a 15-second timeout (roughly 30 seconds worst case, since connect and read are timed separately) so a connection that hangs forever fails the run instead of owning it; that timeout is deliberately excluded from the run's fingerprinted configuration, since it can only affect whether a fetch succeeds, never what a successful fetch means.

5. Modules

Architecture (3) describes how these fit together as a pipeline; this section is the reference -- one line per file on what it is actually responsible for, grouped by directory.

Entry points (outside the module count)

File	Responsibility
main.py	Loads configuration, constructs SignalRouter, runs one symbol/timeframe, returns a process exit code.
live_trading.py	The engine's only market-facing code. Simulates placing an order from a real decision, for observation only -- never sends one.
test_live.py	A three-line manual script: runs the live simulator once and prints where its log landed.
run_tests.py	Dependency-free test runner -- works before pip install has succeeded, which is exactly the condition the dependency test itself checks.

core/ -- orchestration, presentation, and after-the-fact checkability

File	Responsibility
config.py	All tunable constants: market symbol/timeframe, the MEXC base URL and request timeout, structure/indicator lengths, chart settings. Holds no credentials by design.
engine_core.py	Phase7Engine.run() -- the pipeline itself. Calls every other directory in order and assembles the raw state a decision gets built from.
panel_render.py	Renders a finished decision object as the colored terminal panel a person actually reads.
decision_contract.py	Defines the required shape of a decision object and is checked against every run's output before it can reach the panel or the log.
decision_log.py	Writes each decision to a permanent, append-only record.
lineage.py	Hashes every input frame and the run itself, archives the raw candles, and prunes archives on a retention schedule -- so an old decision stays verifiable once its exact market data is gone.
code_fingerprint.py	Hashes the engine's own code (the docstring-stripped parse tree of every production file) so a decision can be tied to the exact code version that produced it.

data/ -- acquisition and integrity

File	Responsibility
data_fetcher.py	Fetches OHLCV from MEXC or a pinned CSV directory; validates every fetch before returning it; hands back an independent copy each call.
validation.py	validate_ohlcv() -- the actual integrity checks: missing/duplicate candles, impossible prices, timestamp ordering, staleness.

indicators/ and structure/ -- feature computation

File	Responsibility
indicators.py	Computes the technical indicators (EMA, RSI, ADX, ATR, VWMA, Supertrend) and handles a failed indicator by dropping its column and reporting the loss, rather than substituting a confident-looking constant.
trend_health.py	Scores overall trend health from the indicator set -- one of the inputs bias_engine combines into direction.
volume_profile.py	Computes the volume profile (high/low-volume nodes) used by structure and by exit-watch proximity checks.
structure.py	StructureEngine -- locates swing highs/lows and the structural sequence (the shape of price action, independent of any single indicator).

models/ -- direction, entry, risk, and the final decision

File	Responsibility
bias_engine.py	calculate_dynamic_bias() -- the sole source of trade direction (raw_bias), and calculate_dynamic_regime() for the market regime/volatility mode.
btc_context.py	Scores BTC correlation and market stress as context only -- structurally unable to cast a second vote on direction.
entry_model.py	Generates entry signals and scores entry quality from bias, trend, and structure.
risk_model.py	RiskModel.calculate_stop_targets() -- the stop and three price targets; read_risk_verdict() reads (never assumes) whether a simulated order passed risk.
exit_model.py	build_exit_watch() -- plain-language advisory flags shown on the panel. The actual exit decision is assembled by signal_router.py from DecisionModel's output, not computed here -- an earlier, larger version of this file was removed when its own outputs were found to reach nothing downstream (see Major Problems, 9).
decision_model.py	DecisionModel._determine_final_action() -- combines bias, trend, entry, and risk into the one final action (LONG / SHORT / NO-TRADE and why), reading direction from bias_engine rather than re-deriving it.
signal_router.py	SignalRouter -- the top-level orchestrator main.py actually calls. Validates the engine's raw output against the decision contract, then routes it to the panel and the log.

utils/

File	Responsibility
plotting.py	plot_engine_chart() -- renders the price chart with entry, stop, and target levels overlaid.
decision_log_backup.py	Takes a dated, committed snapshot of the live decision log.

6. Decision Logic

The final action is decided in `models/decision_model.py`, in a fixed order where an earlier gate can end the run before a later one is even evaluated. Risk is checked first, and its verdict is read rather than assumed: `read_risk_verdict()` returns True, False, or None, and None (no verdict recorded) produces **NO-TRADE (RISK NOT ASSESSED)** -- never a default pass. A recorded failure produces **NO-TRADE (RISK TOO HIGH)**. Only a recorded pass lets the run reach the directional logic at all.

Direction comes from exactly one place: `bias_engine`'s `raw_bias`. A ruling from 5 September 2026 -- "a lean is not a case" -- adds a strength floor on top of the direction itself: a BULLISH or BEARISH lean below `MIN_ACTION_BIAS` (30 of 100) returns WAIT rather than acting on a weak signal. Above that floor, the action graduates through four levels depending on trend health and entry quality against fixed thresholds (`AGGRESSIVE_TREND_HEALTH_MIN = 75`, `AGGRESSIVE_ENTRY_SCORE_MIN = 70`, `CONSERVATIVE_TREND_HEALTH_MIN = 50`): AGGRESSIVE LONG/SHORT when trend, entry quality, and momentum all agree and the risk regime allows it; plain LONG/SHORT when they agree but the risk regime does not; CONSERVATIVE LONG/SHORT when only the macro trend agrees and entry quality is weaker; otherwise WAIT.

The plan-contradiction guard.

On 2 September 2026 the engine printed CONSERVATIVE LONG with a stop above price and three descending targets -- every number correctly computed, the label on it wrong. `_refuse_incoherent_plan()` now checks the action against `_plan_direction()`, which reads direction off the risk plan's own targets (ascending from the stop is a long, descending a short) rather than off any bias field -- a check that asked the same source the action asked could not have caught the two disagreeing. When they disagree, the result is **NO-TRADE (PLAN CONTRADICTS ACTION)**, not a relabelling to whichever side looks right: one of the two sources is wrong, and which one cannot be determined from inside that function, so refusing is the only answer that is certainly not the wrong one.

Confidence is computed after the action, as a real multi-factor score (bias strength, trend health, entry quality, and agreement across them) rather than -- as an earlier version of the engine did -- simply renaming `trend_health` to `confidence_score` and reporting it as if it were independent evidence. When a run is degraded (an indicator failed, or data was rejected and something had to be inferred), a separate rule -- "degrade, don't halt," ruled 29 August 2026 -- lets the run continue with what it could compute, but caps confidence at a fixed ceiling and records that the action would have been different at full confidence; a degraded result never authorizes a trade at full conviction. A BTC-adjusted confidence figure is computed separately again, from the same base confidence and the correlation/stress context, and never feeds back into the action itself -- macro context is read, not voted with.

7. Testing Methodology

Every change is checked three ways before it is considered safe: `pytest` with `pandas_ta` installed (the full path, including every indicator), `pytest` without it (confirming the engine degrades rather than crashes when an optional dependency is missing), and `run_tests.py`, a dependency-free runner that works with nothing but a Python interpreter. That third runner exists because the suite has to prove the engine runs on a clean machine before `pip` install has even succeeded -- which is exactly the `scenario test_declared_dependencies_cover_actual_imports` checks.

The suite's spine is the golden path: `tests/fixtures/` pins a synthetic OHLCV dataset (`TESTUSDT`, generated deterministically from a fixed seed, regenerating byte-identical forever) and `tests/fixtures/golden_decision.json` pins the exact decision object the engine must still produce from it. `test_golden_path.py` runs the real entry point (`SignalRouter.route()`, not a shortcut into the engine) against that fixture and diffs the result against the snapshot -- so a change that alters what the engine decides, not merely how it is written, is caught immediately. The golden path itself was wrong on its first run: it originally called `Phase7Engine.run()` directly, bypassing the router and the decision layer entirely, and the failure exposed that the test, not the engine, had asked the wrong question. Fixed by routing through `SignalRouter` like every real caller does -- recorded rather than quietly corrected, per this project's practice of keeping its own mistakes in the log.

Every fixed defect gets a permanent regression test named for the defect, not the fix -- `test_no_circular_reasoning.py`, `test_no_lookahead.py`, `test_no_fabricated_fallbacks.py`, `test_frame_ownership.py`, `test_execution_surface.py` (five separate guards for Item 18, kept compliant continuously rather than by one-time snapshot) -- so a regression announces itself by name. Negative controls are treated as first-class: `test_accepts_clean_data` exists because a validator that rejects everything passes every rejection test without being a validator at all, and several risk/degradation tests are explicitly named "negative control" for the same reason -- a test suite that can never fail proves nothing.

Results from the sandbox (Linux) are stated as sandbox evidence, not engine truth, until confirmed on Viktor's own machine (Windows) -- a platform-discipline habit adopted after line-ending and path-separator differences between the two produced at least one defect that a Linux-only check would have missed. Every delivered change is verified against a fresh clone before it reaches Viktor, so "it works" always means "it works from a clean checkout," not merely "it works in the tree it was written in."

8. Debugging Process

The most consistent lesson of this project is that running the code finds defects that reading it does not. Four independent audit passes across three AI models read the source and produced findings; a test harness that actually executed the engine found three things none of them had -- including that, as published, the repository did not start from a fresh clone at all. That pattern repeated after remediation: the harness caught a chart renderer silently drawing charts with no price candles, a `NameError` that killed every run, and a test that contradicted the design it was written to check, each on the first run after a change that a static read had passed. A different method beat a different model, consistently enough that "run it and read the panel" is now a standing step before any change is called finished, not an occasional sanity check.

A second habit follows from the first errors Claude made that Viktor caught, on 3 September 2026: state a claim's verification status inside the sentence that makes the claim, not as a caveat after it. That session produced two incomplete change predictions, a defect shipped in a patch, three alarms raised about the project's own documentation without reading the documentation first, and one test assertion that would have passed vacuously -- five errors, all the same shape: confidence stated independent of whether anything had actually been checked. The fix was not to hedge everything, which destroys the same signal as overconfidence does, but to say plainly what was checked and what was not, in the same breath as the conclusion.

A third, related habit: when a document makes a claim about the project's own state -- a commit count, a finding's status, whether a gap exists -- check it against the actual git history or the actual file before repeating it. Claude told Viktor "nine Major and Moderate findings still open" on 1 September 2026, a number read from an audit report's verdicts rather than from the code; checking each location the report quoted found three already closed by unrelated work, and the true count was different. The same habit caught this document's own predecessor problem while it was being written: `README.md`'s status table describes a state from well before round 5 and round 6,

missing four core/ modules that exist now, and was not used as a source here for exactly that reason.

9. Major Problems

Selected from many, these are the defects that most changed how the engine or the process around it works, not simply the ones that were largest to fix.

The direction-source defect (2 September 2026)

The engine printed CONSERVATIVE LONG with a stop above price and three descending targets. Every individual number was computed correctly; the action label was derived from a different reading of direction than the risk plan beneath it used, and nothing checked the two against each other. A trader following the decision line would have bought an instrument the engine had, in the same run, correctly analysed as a short.

Finding 3: a stale value indistinguishable from a real one

Every indicator failure guard asked one question: did the calculation return nothing at all. That catches total failure. It does not catch a series with 299 good values and no value at the exact bar the decision is made on -- because `clean_series()`'s forward-fill had already copied the previous bar's number into that gap before the guard ever ran, so "did it return nothing" was false no matter what happened at the decision bar. Injecting a trailing gap into each indicator in turn found the same blind spot in ATR, RSI, ADX, SuperTrend, and both EMAs -- every one, with no failure recorded and the decision row silently equal to the previous bar. It was never in the original audit's finding list; it was found by reading the audit report closely enough to notice what its own citations implied elsewhere in the code.

Circular reasoning (Item 11)

`trend_health` was counted at least four times and presented on the panel as independently agreeing signals: once directly, once inside `bias_score`, once as a separate confidence bonus, and once more leaking into `bias_engine`'s own reversal/continuation factor. A panel that shows trend, momentum, and validation all agreeing at 95.35 was, in that state, showing one number four times, not three signals confirming each other.

Fabricated fallbacks presented as measurements

A failed indicator substituted a confident-looking constant with no marker: RSI defaulted to 50 (dead centre), ADX to 25 (the trend/no-trend boundary), SuperTrend direction to 1.0 (bullish -- a direction chosen by whoever wrote the fallback). Two of these fallbacks scored the maximum possible points downstream: a missing RSI landed inside the "not extended" band worth full marks, and a missing HVN fell back to the current price, making the distance to it exactly zero and scoring full marks for proximity to a level that was never measured. Elsewhere, a risk fallback returned a stop 1% below price with targets above it regardless of trade direction -- correct for a long, silently inverted and still printed as a real, tradeable plan for a short.

Claims of safety actions that never happened

The panel printed "Trade logged to Logs/phase7_trade_log_<symbol>.csv" on every run. No code anywhere wrote that file. A tool whose entire purpose is producing an inspectable record was asserting that a record had been written when it had not -- rated Critical on the principle that severity reflects consequence, not how much code the repair takes; the fix was one line.

Backtesting broke a previous build

The project-level problem behind all of the above: this is the third build of the engine, and an earlier one was lost to backtesting work attempted before version control, known-good checkpoints, or a fixed evaluation dataset existed to check a change against. A single bad change could not be isolated or rolled back, and took the live engine down with it -- the reason backtesting is deliberately sequenced last in this build rather than early, and the reason the Constitution exists at all.

10. Solutions

The consistent pattern in how these were fixed: change the structure so the mistake cannot recur, rather than add a note asking future work to be careful. Each fix below also became a permanent, named regression test, so a recurrence announces itself rather than waiting to be noticed again by chance.

Problem	Structural fix
Direction-source defect	<code>_refuse_incoherent_plan()</code> reads the risk plan's own targets independently of bias and refuses the trade (NO-TRADE (PLAN CONTRADICTS ACTION)) on any disagreement, rather than trusting either source or guessing which is right.
Finding 3 (stale values)	A single shared guard, <code>indicators.unusable_reason()</code> , that every caller must ask; a value not measured at the decision bar is now classified as failed and routed through the same degradation machinery as a total failure, so no fallback path can bypass it.
Circular reasoning	All duplicate countings of <code>trend_health</code> removed rather than reweighted; <code>bias_score</code> is now the single place all factors combine, and confidence is computed as a real, independent multi-factor score instead of a renamed copy of one input.
Fabricated fallbacks	A failed indicator now drops its column and reports what was lost, instead of substituting a value; a degraded result is capped in confidence and cannot authorize a trade at full conviction ("degrade, don't halt," ruled 29 August 2026).
False safety claims	<code>core/decision_log.py</code> actually writes the record the panel claims exists, and <code>core/decision_contract.py</code> checks every decision object against a required shape before it can reach the panel or the log at all -- an untrue claim can no longer be assembled.
Backtesting risk	Sequencing, not a technical fix: the Constitution defers backtesting until version control, known-good checkpoints, and a fixed evaluation dataset are real rather than aspirational, so a second failure is a recoverable setback instead of a fourth rebuild.

Two artifacts make this pattern durable rather than a one-time cleanup: `core/lineage.py` and `core/code_fingerprint.py`, which hash a run's inputs and the code that produced it respectively, so any future claim about what a past decision was based on can be checked against evidence instead of memory.

11. Results

The engine has not been shown to predict anything, and is not claimed to. One feature (BTC-adjusted prediction) is correctness-validated -- it computes what it was designed to compute -- but empirically unvalidated, and the Constitution requires that distinction be stated plainly wherever the feature appears rather than implied away. What has been produced and checked is a pipeline that runs deterministically on a fixed input, refuses to author a plan it cannot support, and records what it did in a form that can be checked later against the exact code and data that produced it.

The audit history, briefly: the initial ratification audit (four independent runs, models with no prior involvement in the build) found the engine's first Critical defects, including fabricated fallbacks and the false trade-log claim above. A sixteen-item remediation sequence (drawn up by GLM 5.3) was carried out to address that and later review rounds; its final step was an independent re-audit (GPT-5.6 Luna Pro), which found further findings -- including one Critical, on decision-bar integrity, that no prior audit had scheduled -- worked through in batches with rulings recorded for each judgment call. A subsequent round (GPT-6 Astra, round 5) found three fresh Critical Tier-1 defects after that remediation -- measured volatility never reaching the stop/target calculation, entry quality scored for the wrong trade direction, and a stale indicator value presented as current at the decision bar -- each fixed as a separate, individually verified patch. A fix-verification pass on 14 September 2026 (the same auditor confirming its own findings were actually fixed, per Viktor's ruling that this does not require a fresh independent model) confirmed a further five findings fixed, plus one bonus defect it found unprompted and confirmed real -- a harmless duplicate decorator -- fixed the same day.

As of this writing, every finding raised through that round has a landed and independently verified fix, and the test suite passes in all three configurations (pytest with pandas_ta, pytest without it, and the dependency-free runner). Whether that satisfies the release gate's "every Critical fixed and re-audited" bar, and whether the project should be declared portfolio-ready, are calls the project's own record leaves explicitly and repeatedly to Viktor -- this document states what has been verified and does not make either declaration on his behalf.

12. Limitations

The engine has no empirical, predictive validation. Everything checked so far establishes that the engine computes what it claims to compute, correctly and reproducibly -- not that what it computes is a good predictor of anything. That is a deliberate, stated limitation rather than an oversight: the career plan this project serves does not require profitability, and claiming predictive validity without backtesting would be exactly the kind of unsupported confidence the Constitution's Tier 0 purpose forbids.

Backtesting itself has not been rebuilt. It is a real, planned priority, deliberately deferred until the safety net around it -- version control, known-good checkpoints, a fixed evaluation dataset -- is actually in place rather than aspirational, because an earlier attempt at exactly this work destroyed a previous build of the engine. A second failure under the current, better safety net would still be a real cost, not a risk-free one.

Some audit work the project's own record has requested is not yet run: GPT-6 Astra's report named three further, non-Critical investigation runs (BTC-only failure behaviour, fallback equivalence, and how effective the test suite actually is at catching real defects) that remain outstanding as of this writing. In practice, the engine has only been run against one asset (AEROUSDT) and one BTC-correlation pair, even though symbol and timeframe are configuration rather than hardcoded values -- the architecture's generality has not been exercised across a range of pairs.

Project documentation has fallen behind its own code more than once -- README.md's status table and the Engineering Notes log have both, independently, gone stale relative to the actual state of the engine at points in this project's history. The response each time has been a structural fix (documents that build scripts write directly into the repository rather than to a path nobody carries forward) rather than a promise to remember better, but the underlying risk -- that a document is only as current as the last time someone checked it against the code -- is a property of this working method, not something eliminated by any one fix.

13. Lessons Learned

State a claim's verification status inside the claim, not after it. Five errors on 3 September 2026 shared one shape: a conclusion stated with the same confidence whether or not it had actually been checked. The fix adopted was not uniform hedging, which destroys the same signal overconfidence does, but saying plainly, in the sentence making the claim, what had and had not been verified.

Running the code beats reading it. Across this project, a test harness that actually executes the engine has repeatedly found defects that careful static review -- by Claude, by independent AI auditors, or both -- missed, including the repository failing to start from a fresh clone at all. "Read the code and reason about it" and "run the code and look at what it did" are different methods, and this project's evidence is that they find different classes of defect.

Where a mistake cannot be undone, change the structure rather than adding an instruction to be careful. A build script hardcoded to write into /tmp/ could not be fixed by asking future work to remember to copy the file into the repository afterward; it was fixed by giving every build script a shared resolver that always writes into the repository, from any directory, on any platform. The same logic underlies the read-only market guarantee (five independent guards, not one instruction not to trade) and the pinned-data reproducibility mechanism (a missing file is a hard error, not a silent fallback to the live API).

A rename without a safety net is not a small change. Renaming `confidence_score` to `confidence` once broke fourteen downstream modules in this project, because "confidence" had come to mean two different things on two different code paths and nothing forced them to agree. The lesson carried forward is procedural: a decision-object contract now checks the shape of every run's output before it can reach anything downstream, specifically so a field rename or removal fails loudly and immediately rather than silently, three callers away.

Independence, once spent, does not come back. A model that has read the Constitution or an audit package is no longer a neutral judge of either, and this project tracks, by lab and not merely by model version, which reviewers have seen which documents -- because treating a version bump as a fresh reviewer would make the safeguard ceremonial rather than real.

Errors belong in the record, not quietly behind it. This project's engineering log is append-only by rule: a correction to an earlier entry is a new, later-numbered entry that references the old one, never a silent edit. The golden path test being wrong on its own first run, README.md going stale, and the five verification errors above are all still readable in the project's own history for that reason.

14. Future Development

Two goals govern what comes next, deliberately kept apart. Goal A is this portfolio milestone: the release gate open with every Critical fixed and re-audited, remaining findings closed or explicitly accepted as limitations, the Engineering Notes current, this document, the AI-attribution section, and a passing suite against a current golden snapshot. Once all six are true, the plan is to tag that commit (portfolio-v1) as a fixed point -- evidence of what this submission actually was, regardless of what happens next.

Goal B, which explicitly waits behind that tag rather than alongside it, is backtesting: a fixed evaluation dataset, known-good checkpoints, and empirical validation of correctness-validated-but-unvalidated features such as the BTC-adjusted prediction. Because backtesting has already destroyed one build of this engine, the plan treats it as a major architectural effort in its own right once it starts -- built and checkpointed incrementally, with look-ahead bias checked more deliberately than any other invariant, rather than one large attempt in a single pass. With the tag in place first, a failure in that work can break things freely without damaging what was already submitted.

Nearer term, three non-Critical investigation runs GPT-6 Astra's report requested remain open: BTC-only failure behaviour, fallback equivalence, and how effective the test suite actually is at catching real defects rather than merely re-confirming its own prior assertions. A genuinely independent, clean-model audit -- one that has read neither the Constitution nor the code -- is being deliberately saved for the point where independence matters most: immediately before backtesting begins, rather than spent early on a codebase still expected to change.

Beyond the engine itself, Phase-7's role in the wider plan is to be finished, tagged, and put down: the next steps are a practical AI course, a project management course, and automation/IoT fundamentals, with job applications starting once this milestone -- not the eventually-open-ended backtesting work -- has a date attached to it.